

DynaExq: Runtime-Aware Expert Quantization for Efficient MoE Inference under Strict HBM Budgets

Anonymous Submission

Abstract—Mixture-of-Experts (MoE) scales model capacity with bounded per-token compute, but on a single GPU the bottleneck is expert weight residency in device memory. Expert offloading stalls the GPU once per-iteration activation becomes dense; static post-training quantization (PTQ) avoids movement but fixes precision offline, misallocating high-precision capacity when the hot expert set shifts across workloads.

We present DynaExq, a runtime system that manages expert precision as an online residency resource under a fixed memory budget. A routing-driven scheduler identifies hot experts, admits a budget-feasible high-precision set per layer, and triggers promotions/demotions on a non-blocking path: Versioned Expert Residency (VER) with stable handles guarantees the forward pass always sees fully materialized weights, while deterministic pools and admission control prevent transient out-of-memory. On three MoE models across six benchmarks, DynaExq raises accuracy over uniform static PTQ—recovering up to 89% of the INT2-to-INT4 gap on Qwen3-80B (73.09% $\rightarrow$ 77.57%)—and delivers up to 2.73 $\times$ the throughput of offloading baselines at batch size 32.

I. Introduction

Large language models (LLMs) have grown rapidly in capability and deployment [1], [2], [3], [4]. Models based on mixture-of-experts (MoE) [5], [6], [7] are increasingly deployed on single high-end GPUs that must host the full parameter set for low-latency batch serving or for settings without multi-GPU scale-out. In that regime, the practical limit is often device-memory capacity: experts are stored in full so that routing remains unconstrained, and the bottleneck shifts from peak arithmetic throughput to which parameters can remain resident at what precision. Recent MoE LLMs such as Mixtral [8], DeepSeek-V2 [9], and Qwen3 [10] exemplify this trend. Qwen3-MoE-80B, for instance, activates only a few billion parameters per token, yet retaining the full expert pool can require roughly 160 GB of storage—pressing against a fixed memory envelope that must also cover activations, the KV cache, and runtime buffers.

Two classes of techniques are commonly used to fit within such a budget. The first is expert offloading and prefetching [11], [12], [13], [14], [15], [16], [17], [18], [19], which treats GPU memory as a cache and moves experts between the GPU and slower tiers such as host memory or SSD. The second is post-training quantization (PTQ) [20], [21], [22], [23], [24], [25], [26], [27], which compresses expert weights into low bit-width representations. Both are effective in specific regimes, but both rely on assumptions that break under realistic serving mixes.

Offloading works best when each iteration touches a small, stable subset of experts. In practice, MoE activation becomes substantially denser during prefill and at larger batch sizes, expanding the per-iteration working set and increasing transfer pressure (Table II). When the working set outgrows what can be staged over PCIe within the available overlap window, transfers appear as GPU waiting time and amplify tail latency (Figure 1). Even with accurate prediction, the system must still move a large volume of expert weights to sustain throughput.

PTQ reduces expert footprint without transfer dependencies on the critical path. When it avoids online migration entirely, it can achieve the lowest raw per-step latency. However, most MoE PTQ pipelines assign a uniform bit-width to all experts or fix a per-expert precision map offline from limited calibration data—a brittle assumption for MoE. Expert utilization is heavy-tailed over long horizons, and the identity of frequently used experts shifts across workloads (general text, math, code). Under workload shift, a static precision map misallocates scarce high-precision capacity: it either preserves precision for experts that carry little traffic in the current workload, or over-compresses experts that become hot. This paper does not claim to dominate static PTQ on raw latency; it targets a better quality-throughput-memory tradeoff under the same fixed device-memory budget.

We propose DynaExq, a runtime system for single-GPU MoE serving under a hard memory envelope. DynaExq formulates serving as an online, budget-constrained precision allocation problem: expert precision is a dynamically managed residency resource, not only an offline compression choice. The system observes router outputs during serving, estimates which experts carry disproportionate traffic over a stable time horizon, and allocates a limited high-precision budget to those experts while keeping the rest at lower precision so the plan stays feasible under the device memory cap.

The system separates policy from mechanism. On the policy side, a scheduler aggregates routing traces and periodically computes a budget-feasible per-layer resident set for high-precision experts, constrained by a budget that accounts for non-expert parameters and the KV cache. On the mechanism side, DynaExq enforces non-blocking execution by decoupling precision transitions from token computation: stable expert handles resolve to a fully materialized expert version, and promotions and demotions run asynchronously on a dedicated migration

stream. During a transition, the forward pass continues on the last published version; updates become visible only after the data movement completes. Deterministic partitions for expert weights and explicit admission control for promotions keep memory behavior predictable, avoiding allocator jitter and transient out-of-memory failures as the workload evolves.

The main contributions of this work are:

- We formulate single-GPU MoE inference under a strict memory budget as an online precision-residency problem, targeting the quality-throughput-memory tradeoff rather than minimum raw latency alone.
- We define a runtime execution contract: budget-feasible resident sets derived from long-horizon hotness, stable handles with asynchronous publication of new precision versions, and non-blocking forward execution while migrations proceed off the critical path.
- We present a budget-safe memory management design—deterministic expert-weight pools and admission control for promotions—that keeps residency updates predictable without transient OOM as the hot set shifts.
- We evaluate DynaExq on three MoE models (Qwen3-MoE-30B/80B and Phi-3.5-MoE) across six benchmarks: accuracy improves over uniform static PTQ on all three models, with a large recovery over uniform INT2 on Qwen3-80B (73.09% $\rightarrow$ 77.57%; INT4 reference: 78.11%), and throughput reaches up to 2.73 $\times$ that of offloading baselines at batch size 32.

II. Background and Motivation

A. MoE Inference on a Single GPU

MoE layers [5], [6] scale model capacity by routing each token to a top- k subset of expert subnetworks, keeping per-token compute bounded while the full parameter set must remain accessible for unconstrained routing. Modern MoE LLMs [28], [29], [8], [30], [9], [31], [10] can reach hundreds of billions of parameters with only a few billion activated per token. On a single accelerator the bottleneck is therefore device-memory capacity: expert weights dominate the footprint, and whether a model fits depends on how those weights are stored. Expert utilization is also inherently uneven—a small hot set receives a disproportionate share of traffic—so allocating equal fidelity to every expert is rarely cost-effective under a tight memory envelope.

B. Offloading and Prefetching

A family of systems treats GPU memory as a managed cache and fetches experts on demand [11], [12], [13], [14], [15], [16], [17], [18], [19]. These methods work best when each iteration activates a small, stable working set. In practice, activation becomes substantially denser as batch size grows: Tables I and II show the activated-expert ratio per layer exceeding 60% in decode and approaching full activation in prefill. Once the working set outgrows

TABLE I
Expert activation ratio (%) in decode stage.

Model	bs=1	2	4	8	16	32
Qwen3-30B	6.3	11.6	20.1	31.9	46.5	62.0
Qwen3-80B	1.9	3.7	6.7	14.9	25.4	39.2
DeepSeek-V2-Lite	9.0	14.9	23.2	35.9	51.7	67.6

TABLE II
Expert activation ratio (%) in prefill stage.

Model	bs=1	2	4	8	16	32
Qwen3-30B	46.9	60.0	73.4	84.4	92.8	96.6
Qwen3-80B	24.6	35.3	48.8	67.1	78.9	86.2
DeepSeek-V2-Lite	72.9	86.9	94.3	96.0	96.3	96.3

what can be staged within the available overlap window, transfers surface as GPU waiting time (Figure 1).

Observation 1. MoE activation can become dense in prefill and at larger batch sizes, expanding the expert working set beyond what offloading can reliably stage. Frequent evictions and fetches then introduce GPU waiting time.

C. Static Post-Training Quantization

PTQ compresses weights to low bit-widths without retraining. General-purpose schemes such as GPTQ [21], AWQ [20], LLM.int8() [32], ZeroQuant [33], SqueezeLLM [34], and OmniQuant [35] have been extended to MoE-specific settings [22], [23], [24], [25], [26], [27]. Recent extensions to MoE architectures contend with routing-induced imbalance: experts are activated unevenly, calibration data may be biased toward frequently selected experts, and heterogeneous execution interacts with kernel choices and memory traffic. Several MoE-aware pipelines incorporate per-expert or per-block bit allocation, outlier handling, and in some cases system co-design for mixed-precision execution.

PTQ avoids cross-tier movement on the critical path and, absent online migration, achieves the lowest raw per-step latency. But most pipelines fix a uniform or per-expert precision map offline, implicitly assuming expert importance is stable at serving time.

That assumption is brittle for MoE. Expert utilization is heavy-tailed over long horizons and the identity of hot experts shifts across workloads (Figure 2): the top-10 most frequently activated experts under text, math, and code tasks can be entirely disjoint. Under workload shift, a static precision map misallocates scarce high-precision capacity—preserving fidelity for experts that carry little traffic while over-compressing those that become hot.

Activation densification (many experts touched per iteration at large batch) and long-horizon skew (a few experts dominate cumulative traffic) are not contradictory: densification describes concurrent activation within

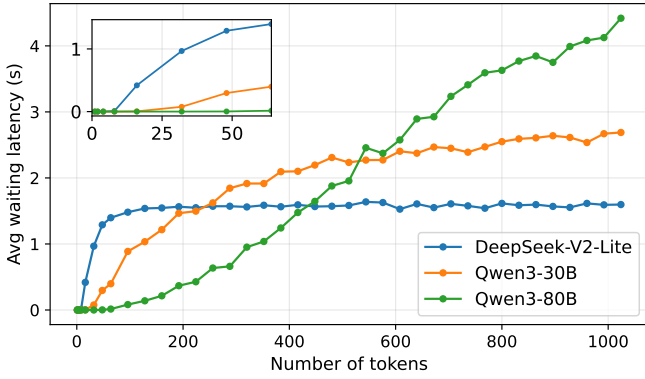


Fig. 1. GPU waiting latency vs. input token count under ExpertFlow on a single A6000.

a step; skew describes traffic concentration across a serving window.

Observation 2. MoE expert usage is heavy-tailed and workload-dependent: a small hot set dominates cumulative invocations, and its identity shifts across tasks. Static precision maps therefore misallocate high-precision capacity under realistic serving mixes.

D. Motivation for Online Precision Control

The observations above suggest adapting expert precision online under a strict GPU budget. A natural concern is whether such adaptation destabilizes quality. We quantize increasing fractions of experts per layer to low precision and measure perplexity on WikiText-2 (128 prompts, 2048 input tokens, 256 output tokens). As Figure 3 shows, when demotion targets infrequently activated experts first, perplexity rises smoothly—protecting the hot set captures a large portion of the quality benefit.

Observation 3. Applying low precision primarily to cold experts yields a smooth, controllable quality-memory curve, suggesting a predictable tradeoff amenable to online management.

E. Challenges for Online Precision Allocation

Online precision control must adapt to routing shift while satisfying constraints that are hard to relax:

- **Hard budget invariance.** Precision changes alter the resident footprint. The system must never violate the device-memory cap; otherwise out-of-memory failures or forced evictions cascade into instability.
- **Critical-path isolation.** Routing decisions occur on the token critical path, but precision transitions involve heavyweight device transfers. Synchronizing them with the forward pass creates stalls that dominate time-to-first-token (TTFT) and time-per-output-token (TPOP).
- **Tail-latency robustness.** Routing is workload-dependent, so naive reallocation can thrash when scores fluctuate, amplifying migration traffic and widening P99 latency.

III. Design: DynaExq

DynaExq treats expert precision as an online, budget-constrained resource. Each expert maintains two weight versions: a low-precision version (b^{lo}) and a high-precision version (b^{hi}). At runtime, the system adapts which experts occupy b^{hi} based on router-observed utilization, subject to three constraints: (C1) budget feasibility, (C2) non-blocking forward path, and (C3) stability under routing fluctuations.

A. Overview and Execution Contract

Figure 4 shows the end-to-end control loop. The design separates policy (what to change) from mechanisms (how to change safely).

a) **Policy.** The scheduler interprets router-derived traffic and chooses a budget-feasible high-precision resident set per layer: hotness estimation via periodic aggregation, top- n selection under fixed per-layer capacities, and hysteresis to damp oscillation. It emits target sets and promotion/demotion candidates but does not allocate GPU memory directly.

b) **Mechanisms.** Three mechanisms implement the serving execution contract: (i) Versioned Expert Residency (VER) exposes stable expert handles and controls when a new precision version becomes visible to kernels; (ii) partitioned device pools and a budget tracker admit or reject each transition before any allocation; (iii) a non-blocking transition pipeline runs data movement on a dedicated CUDA stream and publishes handle updates only after transfers complete. Together, these guarantee that the forward path never blocks on migration, never observes a partially populated weight buffer, and never enters a transition without a prior reservation against the declared HBM envelope.

c) **Worker-side dispatch.** On the worker side, each request invokes the router to obtain top- k expert IDs, which index a handle table. Each handle resolves to an `active_ptr` pointing to a fully materialized expert version on the device. The indirection is the same whether the expert is currently in b^{hi} or b^{lo} ; the kernel calling convention does not change across precision tiers, only the pointer target and associated quantization metadata.

d) **Scheduler-side control.** The scheduler consumes router traces asynchronously. Per-expert hotness statistics feed a budget-feasible selection that determines, for each layer ℓ , a high-precision resident set subject to a layer capacity $n_{\text{hi},\ell}$. The resulting target sets drive a transition manager that maintains promote/evict queues and issues background work on a migration stream.

e) **Budget initialization.** Before inference begins, the system derives per-layer capacities $\{n_{\text{hi},\ell}\}$ from the total device memory M_{total} . Let M_{fixed} denote the memory reserved for non-expert parameters, activations, KV cache, and runtime overhead. Let m_{ℓ}^{hi} and m_{ℓ}^{lo} be the per-expert footprint at each precision tier in layer ℓ (including alignment padding). The initialization solves for the largest

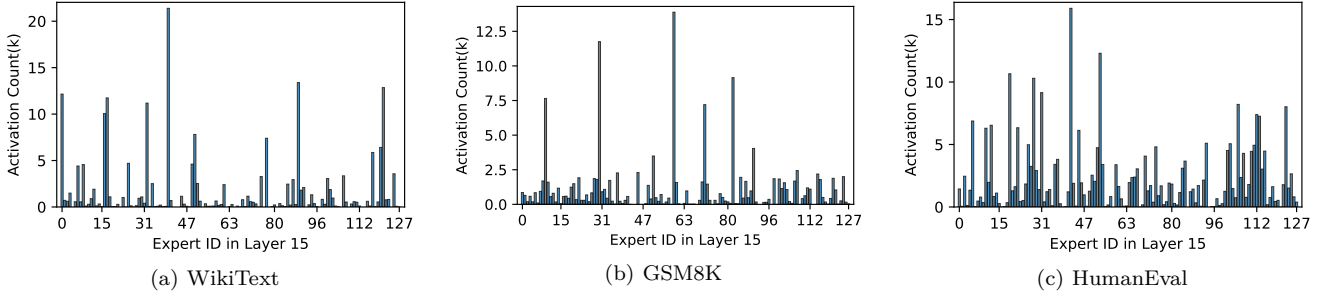


Fig. 2. Expert activation counts under different workloads (Layer 15 of Qwen3-MoE-30B). The top-10 most frequently activated experts are entirely disjoint across text, math, and code tasks.

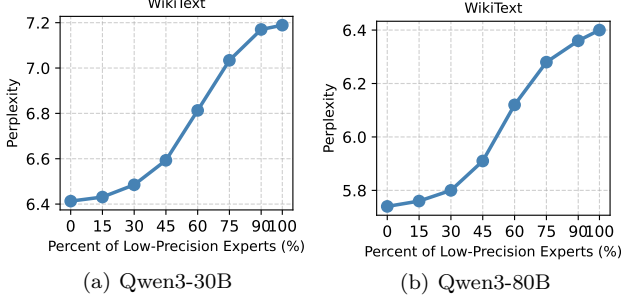


Fig. 3. Perplexity vs. low-precision expert ratio (Qwen3-30B: FP16 vs. INT4; Qwen3-80B: INT4 vs. INT2).

feasible $n_{hi,\ell}$ (typically set uniformly across layers or proportional to layer sensitivity) such that:

$$\sum_{\ell} \left(n_{hi,\ell} \cdot m_{\ell}^{hi} + (E_{\ell} - n_{hi,\ell}) \cdot m_{\ell}^{lo} \right) \leq M_{total} - M_{fixed},$$

where E_{ℓ} is the total number of experts in layer ℓ . A brief warmup phase collects initial hotness scores, and the top- $n_{hi,\ell}$ experts per layer populate the initial high-precision set. After initialization, $n_{hi,\ell}$ remains fixed; only expert identities change online, preserving the budget invariant throughout serving.

Memory is partitioned into separate pools for high- and low-precision versions, isolated from the KV cache region, so that fragmentation in one tier does not consume address space in the other.

B. Online Scheduling Policy

The scheduling policy determines which experts should occupy the limited high-precision capacity. It is kept simple so that its behavior is easy to reason about and its per-tick cost stays small relative to inference.

a) Hotness estimation.: For each MoE layer ℓ and expert e , the runtime maintains a counter $c_{\ell,e}$ that accumulates how many times (ℓ, e) is selected by the router during the current update interval. Updates are triggered with a fixed wall-clock period T_u . A time-based interval gives stable scheduling cadence regardless of batch composition or prompt length. At each tick, a smoothed hotness score is updated via exponential moving average (EMA):

$$S_{\ell,e} \leftarrow \alpha \cdot S_{\ell,e} + (1 - \alpha) \cdot c_{\ell,e},$$

where $\alpha \in [0, 1)$ controls how quickly past observations decay. Counters reset after each update. This construction uses router outputs only and requires no labels or online quality signals.

b) Budget-feasible selection.: For each layer ℓ with capacity $n_{hi,\ell}$, the high-precision set is:

$$H_{\ell} \leftarrow \text{TopN}(\{S_{\ell,e}\}_e, n_{hi,\ell}).$$

Selection is local to each layer and budget-feasible by construction: no cross-layer coordination is needed. The set difference against the current resident set yields promotion candidates $H_{\ell} \setminus H_{\ell}^{cur}$ and demotion candidates $H_{\ell}^{cur} \setminus H_{\ell}$. These candidates are passed to the transition pipeline, which admits promotions only when the budget tracker can reserve the required memory (Section III-D).

c) Hysteresis for stability.: A plain top- N rule can cause unnecessary churn when several experts have similar hotness scores. Each swap triggers a promotion and a matching demotion, consuming migration bandwidth without improving quality. We therefore apply hysteresis: an expert is promoted only if its score exceeds that of the lowest-ranked incumbent by a margin δ , and evicted only if it falls below the highest-ranked outsider by the same margin. This margin can be expressed as an additive threshold on the score or as a rank slack. Hysteresis does not change the budget constraint; it reduces oscillations and makes the transition rate more predictable under transient routing fluctuations.

The policy produces a target set per layer at cadence T_u . Between ticks, the compute path uses the last published handle mapping (window-level pinning) until the transition pipeline finishes any pending version switch.

C. Versioned Expert Residency (VER)

VER defines how experts are represented on the device, how handles stay stable across precision changes, and when a new version becomes visible to kernels. It is a mechanism, independent of the specific hotness rule that selected the target set.

a) Expert entries and stable handles.: Each expert is represented by an expert entry that owns metadata for all supported versions and exports a stable handle to the MoE kernel. The handle is immutable in identity but contains a mutable pointer to the currently active device version.

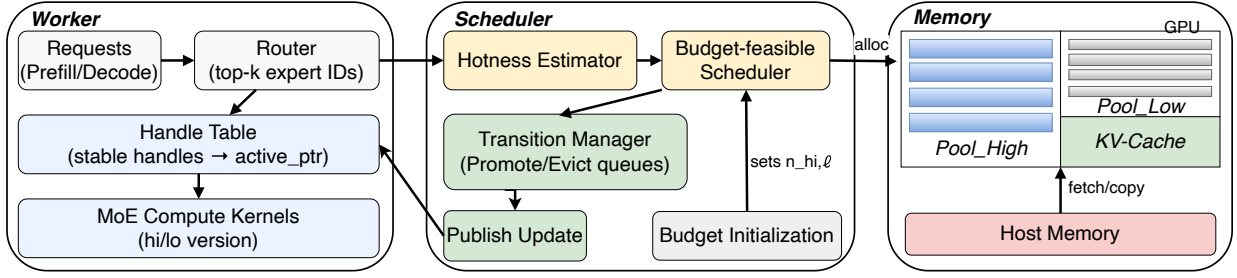


Fig. 4. System architecture of DynaExq. The worker executes MoE inference via stable handles; a scheduler observes router traces and updates expert precision residency under a strict HBM budget; a transition pipeline carries out promotions and demotions on a dedicated migration stream.

The compute path resolves the handle to obtain the active pointer and the associated quantization parameters, then invokes the corresponding kernel variant. This indirection lets VER change residency by updating a pointer while the handle address itself never moves.

b) Residency lifecycle states.: Each expert entry tracks its residency through five mutually exclusive states:

- Resident-Hi: high-precision version is on the GPU; the handle points to it.
- Resident-Lo: only the low-precision version is on the GPU; the handle points to it.
- Promoting: a high-precision buffer is being populated asynchronously; the handle still points to the last complete version (typically the low-precision one) until the copy finishes.
- Demoting: a low-precision buffer is being written to replace the high-precision resident; the handle still points to the current valid version until the new buffer is ready.
- Evicting: a superseded buffer is being freed; the handle already references a different complete version.

One invariant holds across all states: the handle always resolves to a complete, usable weight tensor. No intermediate state exposes a partially filled buffer to the compute path. This is why the forward pass can proceed without blocking, even when promotions or demotions are in flight.

c) Publication semantics.: VER decouples version transitions from the token critical path. When a promotion is initiated, the forward pass continues to read the current version through the stable handle. After the asynchronous copy on the migration stream completes, VER atomically updates the handle’s `active_ptr` so that the next dispatch uses the new version. We call this publish-after-complete: no kernel ever observes a buffer whose contents are still being transferred.

Eviction follows the opposite order: the handle is first repointed to the surviving version (which is already on the device), and only then is the superseded buffer returned to the pool’s free list. This ordering prevents any window in which the handle references freed memory.

D. GPU Memory Management

Dynamic residency stresses GPU memory in two ways. First, promotions and evictions allocate large weight buffers; if these share a general-purpose device heap with variable request sizes, external fragmentation and allocator jitter can inflate latency variance. Second, several transitions may overlap in time; if their aggregate footprint is unbounded, concurrent promotions can exhaust the device budget even when each is individually feasible.

a) Partitioned pools and fragmentation control.: We divide the GPU memory assigned to expert weights into two disjoint pools: `pool_hi` for high-precision versions and `pool_lo` for low-precision versions. Both pools are isolated from the KV cache and framework buffers. Within each pool, allocation uses fixed-size blocks whose granularity matches a single expert’s footprint (with alignment padding). Fixed-block allocation—analogue to the paged memory used for KV caches in serving engines [36]—bounds internal fragmentation to a known fraction of one block and keeps the free/allocate path uniform. Each pool maintains its own free list, so the hot allocation path never invokes the global device allocator. Separating pools by precision tier means that heavy churn in one tier—say, rapid promotions filling `pool_hi`—cannot consume address space that `pool_lo` depends on for its resident set.

b) Budget model and admission.: A BudgetTracker enforces the global memory budget through explicit reservation and release operations. The usable device memory M_{total} is partitioned at startup into:

- M_{fixed} : KV cache, non-expert model parameters, activations, and runtime overhead.
- $M_{\text{exp,hi}}^{\text{cap}}$: cap on resident high-precision expert weights.
- $M_{\text{exp,lo}}^{\text{cap}}$: cap on resident low-precision expert weights.
- A bounded staging region for in-flight transfer buffers, so that concurrent migration work is accounted for before it starts.

Every promotion must call `try_reserve( $b_{\text{hi}}$ )` before entering the transition pipeline. A successful reservation accounts for the footprint against $M_{\text{exp,hi}}^{\text{cap}}$, including any paired release from a demotion already planned in the same transition batch. If the reservation fails, the runtime defers the promotion or schedules evictions to reclaim

TABLE III
Configuration of evaluated MoE models.

	Qwen3-30B	Qwen3-80B	Phi-3.5-MoE
Total Parameters	30.5B	80B	42B
Activated / Token	3.3B	3B	6.6B
Total Weight Size	57 GB	41 GB	78 GB
Expert Weight Fraction	95%	93%	96%
Layers	48	48	32
Experts / Layer	128	512	16
Top- K	8	10	2

d) Host memory.: Expert weights are prepared offline into high- and low-precision kernel-ready layouts and stored in pinned host memory. Retaining both layouts trades host DRAM for predictable promotion latency: no decode or repack on the serving path.

V. Evaluation

We evaluate DynaExq on a single GPU under a fixed device-memory budget. The objective is not minimum raw per-step latency but a better quality-throughput-memory tradeoff within that budget. The experiments address four questions: (i) quality at comparable resident footprint; (ii) latency and throughput when activation becomes dense; (iii) how static PTQ and DynaExq compare on the latency axis; and (iv) what the present experiments do and do not cover (Sec. V-D).

A. Experimental Setup

Models. We use Qwen3-30B-A3B-Instruct, Qwen3-80B-A3B-Instruct [10], and Phi-3.5-MoE-Instruct [38] (Table III).

Benchmarks. WikiText-2 [39] (perplexity), MMLU-Pro [40], GPQA [41], AIME25 [42], GSM8K [43], and HumanEval [44].

Hardware and stack. All experiments run on a single NVIDIA RTX A6000 (48 GB device memory). DynaExq is integrated into a PyTorch/Transformers-based MoE inference stack [45], [37]. Hot experts use a higher-precision tier (FP16 for Qwen3-30B and Phi-3.5-MoE; INT4 for Qwen3-80B); cold experts remain at a lower tier (INT4 or INT2 respectively).

Baseline selection principle. Static PTQ baselines are chosen so the full expert pool fits in device memory alongside non-expert weights and runtime allocations—the same constraint DynaExq operates under. For Qwen3-30B and Phi-3.5-MoE the strongest uniform static point within the single-GPU footprint is INT4. For Qwen3-80B we report two static references: uniform INT4 (higher accuracy, tighter fit) and uniform INT2 (more aggressive compression that leaves headroom for a mixed-precision policy). ExpertFlow [19] serves as the offloading/prefetch baseline.

B. Quality under a Fixed Memory Budget

Table IV summarizes aggregate accuracy. Results are interpreted per model rather than across a single ordering, because the feasible static baselines differ.

Qwen3-30B. Uniform INT4 drops the average from 65.29 (FP16) to 63.98. DynaExq brings it back to 64.38, recovering roughly 30% of the FP16 gap by concentrating higher precision on frequently routed experts while keeping most weights in the low tier.

Qwen3-80B. Against uniform INT2 (73.09), DynaExq reaches 77.57—recovering 89% of the gap toward INT4 (78.11). This is the case where a mixed-precision policy has the most room to work, because the static uniform baseline is aggressively compressed. DynaExq does not strictly dominate the INT4 reference; its strength is improving quality substantially over INT2 at comparable device-memory cost.

Phi-3.5-MoE. INT4 lowers the average from 58.06 to 56.31; DynaExq reaches 57.00, recovering roughly 39% of the gap.

On Qwen3-30B and Phi-3.5-MoE the recovery is moderate, consistent with the INT4 baseline already being a relatively mild compression point. On Qwen3-80B the recovery is large, reflecting more headroom when the baseline is more aggressive. Across all three models, DynaExq consistently sits between the uniform low-bit baseline and the higher-precision reference, which is the intended operating point for a hotness-driven policy under a fixed budget.

C. Latency and Throughput

We compare DynaExq against static PTQ and ExpertFlow, sweeping batch size and prompt length to stress regimes where MoE activation becomes less sparse.

a) Why static PTQ shows the lowest latency.: Static PTQ keeps a single precision map with no ongoing promotion/demotion and no host-to-device copies for precision switches. Its latency advantage is structural, not an artifact of our setup. The paper’s contribution targets the multi-dimensional tradeoff, not single-metric latency.

b) TTFT.: Figure 6 shows TTFT vs. batch size. Static PTQ is lowest throughout. ExpertFlow rises sharply in both average and P99 as batch grows—prefill activates more experts per iteration and exposes PCIe movement as visible waiting once the working set outgrows the overlap window. DynaExq tracks static PTQ more closely than ExpertFlow and stays far below ExpertFlow across batch sizes, with the gap widening when prefill is denser.

c) TPOP.: Figure 7 reports TPOP during decode. ExpertFlow shows higher TPOP and a widening tail as batch increases. DynaExq reduces transfer interference by using resident tiers and a separate migration stream; its TPOP stays closer to static PTQ, with a smaller average-to-P99 spread than ExpertFlow.

TABLE IV
Accuracy comparison across models and methods under comparable device-memory budgets.

Model	Method	MMLU-Pro	GPQA	AIME25	GSM8K	HumanEval	Avg.
Qwen3-MoE-30B	FP16	73.37	54.55	23.33	90.45	84.76	65.29
	INT4	72.84	53.54	20.00	89.39	84.15	63.98
	DynaExq	73.09	54.14	20.00	89.91	84.76	64.38
Qwen3-MoE-80B	INT4	75.92	72.22	70.00	87.04	85.37	78.11
	INT2	72.75	66.67	63.33	80.97	81.71	73.09
	DynaExq	75.48	71.21	66.67	86.73	84.76	77.57
Phi-3.5-MoE	FP16	54.23	36.75	40.00	88.66	70.64	58.06
	INT4	53.40	34.52	36.67	87.54	69.41	56.31
	DynaExq	53.91	35.94	36.67	88.25	70.22	57.00

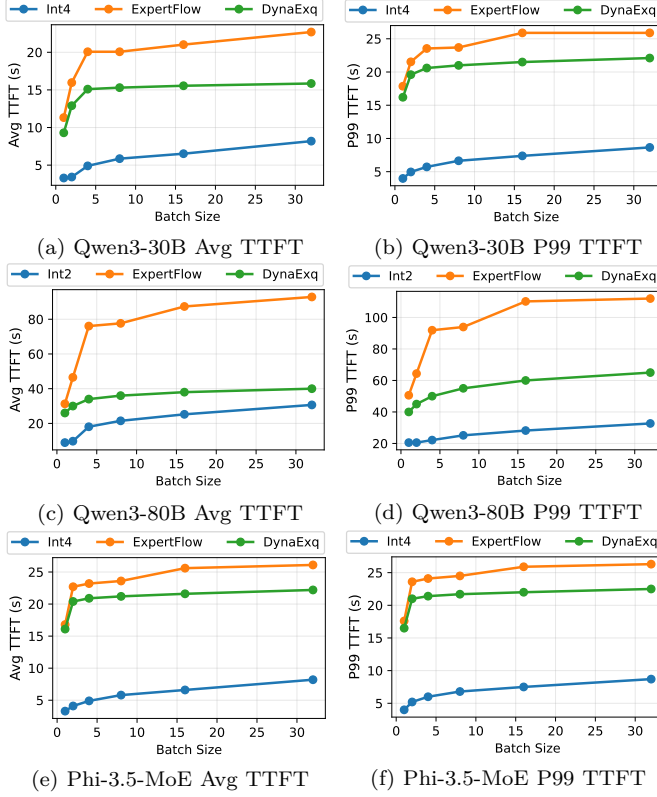


Fig. 6. TTFT (average and P99) vs. batch size.

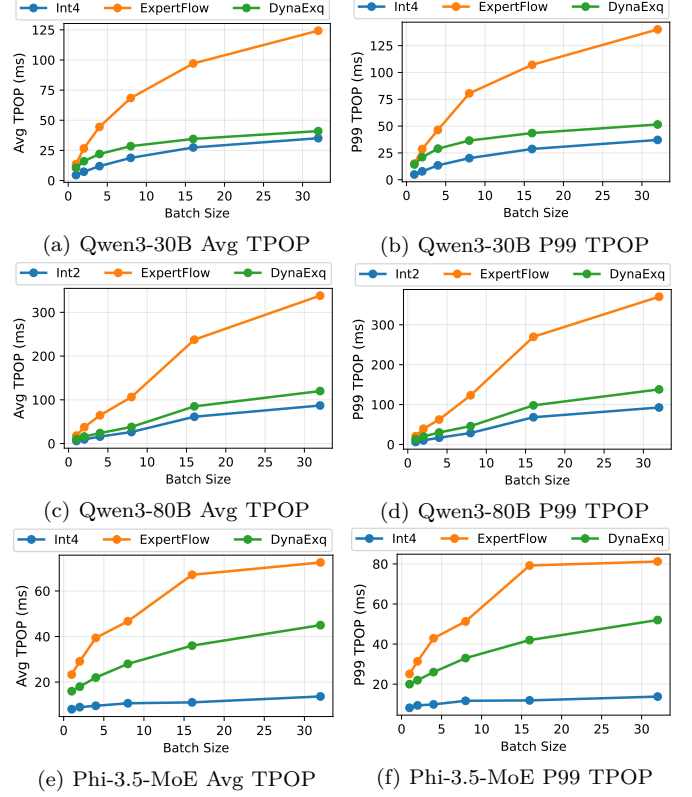


Fig. 7. TPOP (average and P99) vs. batch size.

d) End-to-end latency.: Figure 8 mirrors the per-phase trends: static PTQ lowest, ExpertFlow highest, DynaExq in between and closer to static PTQ. As batch size grows, ExpertFlow compounds transfer costs in both average and P99; DynaExq avoids blocking the forward path on transitions and keeps background interference bounded.

e) Throughput.: Figure 9 reports tokens/s vs. batch size. DynaExq outperforms ExpertFlow by $1.42\text{--}2.73\times$ across the three models; the gap widens at larger batch sizes. Dense prefill reduces overlap opportunities for off-loading, making expert movement a throughput bottleneck; DynaExq keeps a bounded resident set within budget and avoids blocking on publication, so throughput scales more favorably.

f) Latency scaling with prompt length.: Figure 10 sweeps tokens per request and reports TTFT (average and P99). Latency grows quickly for short prompts and plateaus once prefill work dominates. Static PTQ stays lowest and varies little with prompt length. ExpertFlow shows the steepest growth and the widest tail as prefill becomes less sparse. On Qwen3-30B, ExpertFlow’s average TTFT rises to around 10s at the longest prompts while DynaExq stabilizes near mid-single digits with P99 well below ExpertFlow. On Qwen3-80B, ExpertFlow converges to roughly 40–50s in average TTFT; DynaExq remains substantially lower. Phi-3.5-MoE follows the same pattern. Longer prompts increase expert movement volume; DynaExq limits migration-stream interference relative to ExpertFlow, not relative to static PTQ’s zero-migration

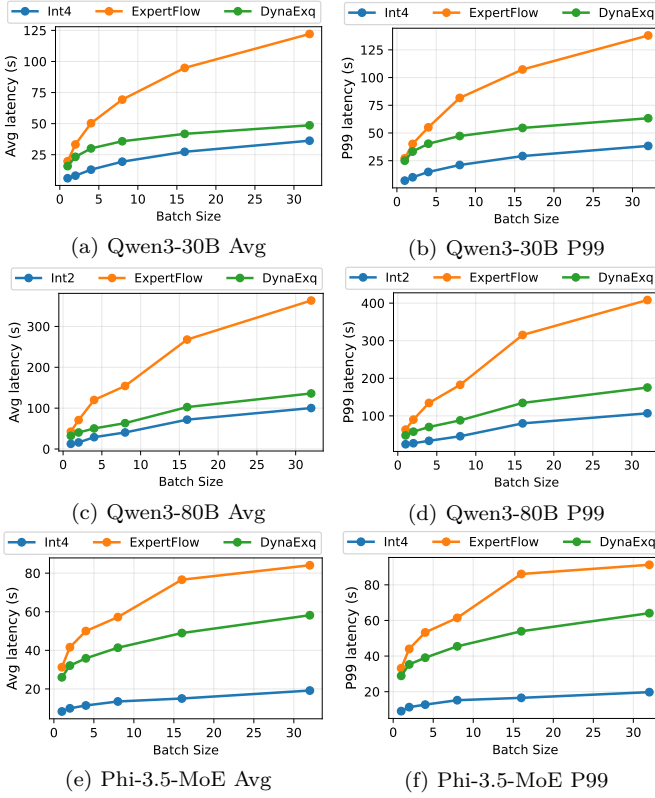


Fig. 8. End-to-end latency (average and P99) vs. batch size.

path.

D. Scope and Limitations

What is covered. End-to-end quality, latency, throughput, and tail behavior under fixed memory budgets across three MoE models and six benchmarks, compared against static PTQ and one offloading baseline.

What is not. We do not report microbenchmarks for migration counts, transferred bytes, host memory overhead, peak device memory, scheduler CPU time, or fragmentation proxies. Memory predictability and component-level overhead remain design claims, not empirically measured properties. We also do not include sweeps over T_u , α , hysteresis margin, high-precision set size, or ablations disabling individual components (VER, async publication, the online scheduler). Such studies would clarify sensitivity and isolate contributions, but are outside the scope of the present experiments.

Baselines. ExpertFlow is the only offloading system compared end-to-end. HOBBIT [17], MxMoE [23], ProMoE [18], and MoE-Infinity [13] are discussed in Related Work but not benchmarked, because a fair comparison under identical single-GPU budget and workload conditions would require substantial re-implementation.

Platform and framework. All experiments use a single A6000 (48 GB) on a HuggingFace Transformers stack rather than a production framework such as vLLM [36] or SGLang [46]. This GPU represents a memory-bound regime where expert weights fill or exceed device memory,

but we have not validated the design on other GPU classes or memory sizes.

VI. Related Work

We organize prior work by the assumption each line makes about expert weights at inference time. The distinction matters because our contribution is not the observation that hot experts justify higher precision—that pattern appears across MoE quantization and hybrid offload work—but the serving execution contract that makes online precision residency budget-safe and non-blocking on a single GPU.

A. MoE Serving, Offloading, and Prefetch

Systems such as DeepSpeed-MoE [47], DeepSpeed Inference [48], Tutel [49], and MegaBlocks [50] optimize routing, grouping, and kernel utilization for experts already resident at fixed precision. General-purpose LLM serving engines such as vLLM [36], Orca [51], and SGLang [46] manage KV-cache memory and request scheduling but do not address MoE-specific expert residency. FlexGen [52] demonstrated offloading for dense LLM inference on a single GPU. For MoE specifically, a complementary line treats GPU memory as a cache: Adap-gating [11], EdgeMoE [12], MoE-Infinity [13], Mixtral-offloading [14], Pre-gated MoE [15], AdaMoE [16], ProMoE [18], ExpertFlow [19], and others [53], [54] differ in prediction, staging, and coordination but treat each expert as a single-precision artifact whose placement and fetch time are the control variables. They address where an expert resides and when it is loaded, not how a runtime admits concurrent precision tiers, publishes version switches without stalling the forward path, or enforces a strict budget for mixed-precision residency.

B. Static PTQ and Generic Low-Bit Inference

Weight-only and activation-aware schemes—GPTQ [21], AWQ [20], LLM.int8() [32], SmoothQuant [55], ZeroQuant [33], SpQR [56], SqueezeLLM [34], AQLM [57], OmniQuant [35], and QuaRot [58]—improve accuracy at aggressive bitwidths but produce a static precision configuration. Adaptive PTQ methods (HAQ [59], FlexRound [60]) adjust bit choices at calibration time, yet the deployed graph is still fixed. Neither line addresses MoE-specific online residency under a hard per-device envelope with sparse, shifting activation.

C. MoE-Specific Mixed Precision

MxMoE [23] derives mixed-precision configurations from expert sensitivity and couples them with mixed-precision grouped GEMMs. MoPEQ [61] assigns bitwidths using activation frequency and sensitivity measures. Earlier work on stable MoE design [62], [28] and expert specialization [30] explored how expert heterogeneity affects model quality, providing motivation for non-uniform precision allocation. These methods improve the quality-memory tradeoff at model-build time, but the resulting

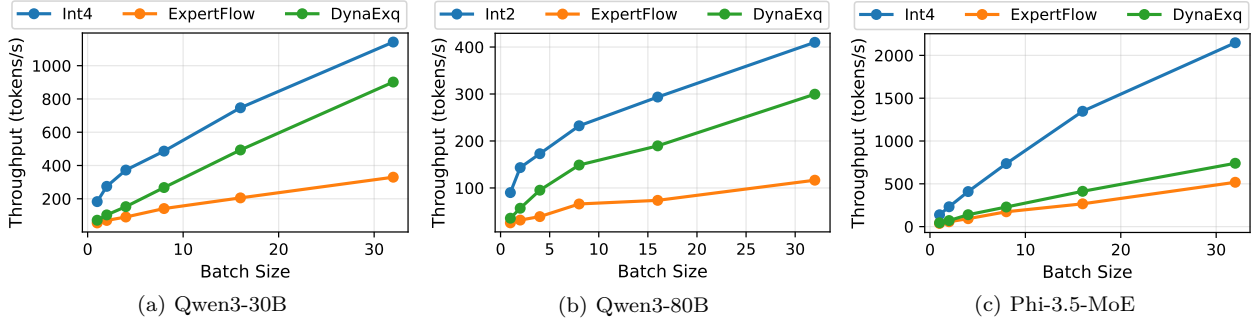


Fig. 9. End-to-end throughput (tokens/s) vs. batch size.

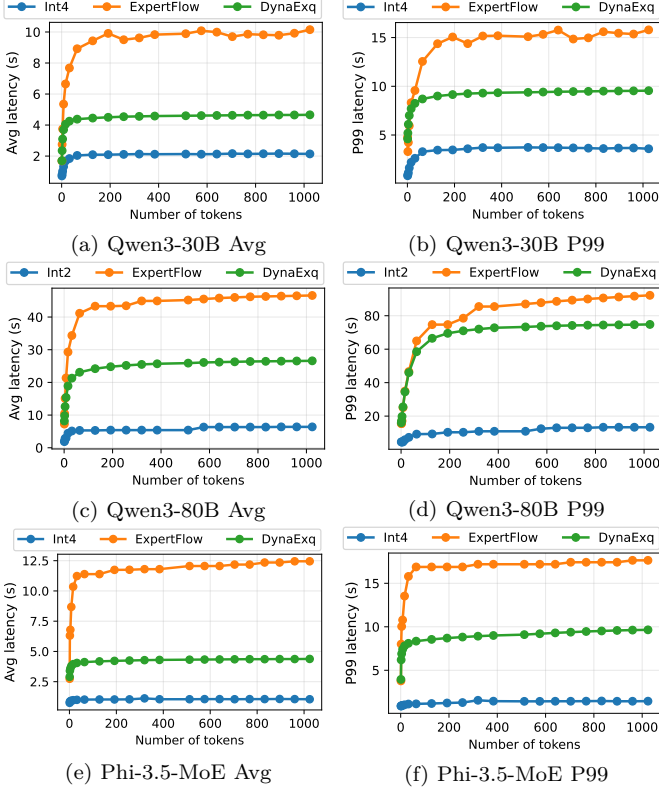


Fig. 10. TTFT vs. prompt length (average and P99).

precision map stays fixed during serving and does not adapt as routing distributions shift.

D. Hybrid Systems

HOBBIT [17] combines mixed-precision expert representations with offloading to reduce load cost for cache-miss paths. It demonstrates that precision heterogeneity and memory hierarchy policies interact, but differs from DynaExq in scope and mechanism. DynaExq does not optimize offloading latency; it defines a single-GPU serving contract for online residency: (i) stable handles with versioned publication so precision transitions never expose partial weights, (ii) admission control before allocating high-precision resident capacity, and (iii) deterministic pools to bound fragmentation. These mechanisms are

TABLE V
Positioning of DynaExq against closely related MoE systems.

System	Primary focus relative to DynaExq
MoE-Infinity [13]	Locality-aware offload/cache of fixed-precision experts; no versioned precision residency or publication semantics.
ProMoE [18]	Proactive prefetch/caching; movement scheduling for fixed-precision weights, not online precision tier changes.
MxMoE [23]	Offline mixed-precision PTQ and kernels; static assignment, no runtime residency control.
HOBBIT [17]	Mixed-precision offloading to reduce miss cost; does not specify non-blocking publication, budget admission, or pool-based memory for online residency.

complementary to, and distinct from, offloading-aware mixed precision.

E. Positioning DynaExq

Table V condenses how each closely related system compares to DynaExq’s execution contract.

VII. Conclusion

We presented DynaExq, a runtime system for single-GPU MoE inference that manages expert precision as an online residency resource. A routing-driven scheduler selects hot experts under a fixed memory budget; Versioned Expert Residency decouples precision transitions from the forward path; deterministic pools and admission control keep memory usage predictable. On Qwen3-30B/80B and Phi-3.5-MoE, DynaExq recovers 30–39% of the FP16 gap over INT4 and 89% of the INT2-to-INT4 gap, while reaching up to 2.73× the throughput of offloading baselines.

The current evaluation lacks ablation studies, overhead microbenchmarks, and comparisons with mixed-precision systems such as HOBBIT and MxMoE. All experiments use a single A6000 on a research-grade inference stack. Extending to multi-GPU settings and production frameworks are natural next steps.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar et al., "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [3] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan et al., "The llama 3 herd of models," *arXiv e-prints*, pp. arXiv-2407, 2024.
- [4] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong et al., "A survey of large language models," *arXiv preprint arXiv:2303.18223*, vol. 1, no. 2, 2023.
- [5] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [6] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.
- [7] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Zhao, A. M. Dai, Q. V. Le, J. Laudon et al., "Mixture-of-experts with expert choice routing," *Advances in Neural Information Processing Systems*, vol. 35, pp. 7103–7114, 2022.
- [8] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
- [9] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.
- [10] Q. Team, "Qwen3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [11] J. Li, Q. Su, Y. Yang, Y. Jiang, C. Wang, and H. Xu, "Adaptive gating in mixture-of-experts based language models," *arXiv preprint arXiv:2310.07188*, 2023.
- [12] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, "Edgemoe: Fast on-device inference of moe-based large language models," *arXiv preprint arXiv:2308.14352*, 2023.
- [13] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "Moe-infinity: Offloading-efficient moe model serving," *arXiv preprint arXiv:2401.14361*, 2024.
- [14] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.
- [15] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1018–1031.
- [16] S. Zhong, L. Liang, Y. Wang, R. Wang, R. Huang, and M. Li, "Adapmoe: Adaptive sensitivity-based expert gating and management for efficient moe inference," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.
- [17] P. Tang, J. Liu, X. Hou, Y. Pu, J. Wang, P.-A. Heng, C. Li, and M. Guo, "Hobbit: A mixed precision expert offloading system for fast moe inference," *arXiv preprint arXiv:2411.01433*, 2024.
- [18] X. Song, Z. Zhong, R. Chen, and H. Chen, "Promoe: Fast moe-based llm serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.
- [19] Z. Shen, K. Chu, Y. Zhang, D. Xiang, R. Wu, and W. Zhang, "Expertflow: Adaptive expert scheduling and memory coordination for efficient moe inference," 2025. [Online]. Available: <https://arxiv.org/abs/2510.26730>
- [20] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.
- [21] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [22] Y. J. Kim, R. Fahim, and H. H. Awadalla, "Mixture of quantized experts (moqe): Complementary effect of low-bit quantization and robustness," 2023. [Online]. Available: <https://arxiv.org/abs/2310.02410>
- [23] H. Duanmu, X. Li, Z. Yuan, S. Zheng, J. Duan, X. Zhang, and D. Lin, "Mxmoe: Mixed-precision quantization for moe with accuracy and performance co-design," 2025. [Online]. Available: <https://arxiv.org/abs/2505.05799>
- [24] Z. Fu, N. Ding, K. Han, X. Yu, X. Li, X. Chen, Y. Tang, and Y. Wang, "Eaquant: Enhancing post-training quantization for moe models via expert-aware optimization," 2025. [Online]. Available: <https://arxiv.org/abs/2506.13329>
- [25] E. Frantar and D. Alistarh, "Qmoe: Practical sub-1-bit compression of trillion-parameter models," 2023. [Online]. Available: <https://arxiv.org/abs/2310.16795>
- [26] J. Zhang, Y. Zhang, B. Zhang, Z. Liu, and D. Cheng, "Moqe: Improve quantization model performance via mixture of quantization experts," *arXiv preprint arXiv:2508.09204*, 2025.
- [27] X. Hu, Z. Chen, D. Yang, Z. Xu, C. Xu, Z. Yuan, S. Zhou, and J. Yu, "Moequant: Enhancing quantization for mixture-of-experts large language models via expert-balanced sampling and affinity guidance," 2025. [Online]. Available: <https://arxiv.org/abs/2505.03804>
- [28] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [29] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations*, 2021.
- [30] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu et al., "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.
- [31] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan et al., "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [32] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 30 318–30 332.
- [33] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 168–27 183, 2022.
- [34] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, "SqueezeLLM: Dense-and-sparse quantization," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 1–14, 2024.
- [35] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "OmniQuant: Omnidirectionally calibrated quantization for large language models," in *International Conference on Learning Representations*, 2024.
- [36] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [37] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz et al., "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [38] M. Abidin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, A. Benhaim, M. Bilenko, J. Bjorck, S. Bubeck, M. Cai, Q. Cai, V. Chaudhary, D. Chen, D. Chen, W. Chen, Y.-C. Chen, Y.-L. Chen, H. Cheng, P. Chopra, X. Dai, M. Dixon, R. Eldan, V. Fragoso, J. Gao, M. Gao, M. Gao, A. Garg, A. D. Giorno, A. Goswami, S. Gunasekar, E. Haider, J. Hao, R. J. Hewett,

- W. Hu, J. Huynh, D. Iter, S. A. Jacobs, M. Javaheripi, X. Jin, N. Karampatziakis, P. Kauffmann, M. Khademi, D. Kim, Y. J. Kim, L. Kurilenko, J. R. Lee, Y. T. Lee, Y. Li, Y. Li, C. Liang, L. Liden, X. Lin, Z. Lin, C. Liu, L. Liu, M. Liu, W. Liu, X. Liu, C. Luo, P. Madan, A. Mahmoudzadeh, D. Majercak, M. Mazzola, C. C. T. Mendes, A. Mitra, H. Modi, A. Nguyen, B. Norick, B. Patra, D. Perez-Becker, T. Portet, R. Pryzant, H. Qin, M. Radmilac, L. Ren, G. de Rosa, C. Rosset, S. Roy, O. Ruwase, O. Saarikivi, A. Saied, A. Salim, M. Santacrose, S. Shah, N. Shang, H. Sharma, Y. Shen, S. Shukla, X. Song, M. Tanaka, A. Tupini, P. Vaddamanu, C. Wang, G. Wang, L. Wang, S. Wang, X. Wang, Y. Wang, R. Ward, W. Wen, P. Witte, H. Wu, X. Wu, M. Wyatt, B. Xiao, C. Xu, J. Xu, W. Xu, J. Xue, S. Yadav, F. Yang, J. Yang, Y. Yang, Z. Yang, D. Yu, L. Yuan, C. Zhang, C. Zhang, J. Zhang, L. L. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, and X. Zhou, "Phi-3 technical report: A highly capable language model locally on your phone," 2024. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [39] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," 2016.
- [40] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang et al., "Mmlu-pro: A more robust and challenging multi-task language understanding benchmark," *Advances in Neural Information Processing Systems*, vol. 37, pp. 95 266–95 290, 2024.
- [41] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman, "Gpqa: A graduate-level google-proof q&a benchmark," in *First Conference on Language Modeling*, 2024.
- [42] M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev, "Matharena: Evaluating llms on uncontaminated math competitions," Feb. 2025. [Online]. Available: <https://matharena.ai/>
- [43] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.
- [44] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021.
- [45] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga et al., "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.
- [46] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kober, L. Shi, C. Wu et al., "SGLang: Efficient execution of structured language model programs," *arXiv preprint arXiv:2312.07104*, 2024.
- [47] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *International conference on machine learning*. PMLR, 2022, pp. 18 332–18 346.
- [48] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "DeepSpeed Inference: Enabling efficient inference of transformer models at unprecedented scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.
- [49] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram et al., "Tutel: Adaptive mixture-of-experts at scale," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 269–287, 2023.
- [50] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "Megablocks: Efficient sparse training with mixture-of-experts," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 288–304, 2023.
- [51] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation*, 2022, pp. 521–538.
- [52] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 1–16, 2023.
- [53] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "Swapmoe: Serving off-the-shelf moe-based large language models with tunable memory budget," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 6710–6720.
- [54] H. Yu, X. Cui, H. Zhang, H. Wang, and H. Wang, "fmoe: fine-grained expert offloading for large mixture-of-experts serving," *arXiv e-prints*, pp. arXiv-2502, 2025.
- [55] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International conference on machine learning*. PMLR, 2023, pp. 38 087–38 099.
- [56] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefer, and D. Alistarh, "Spqr: A sparse-quantized representation for near-lossless llm weight compression," *arXiv preprint arXiv:2306.03078*, 2023.
- [57] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, "Extreme compression of large language models via additive quantization," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 1–13, 2024.
- [58] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefer, and J. Hensman, "Quarot: Outlier-free 4-bit inference in rotated llms," *Advances in Neural Information Processing Systems*, vol. 37, pp. 100 213–100 240, 2024.
- [59] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "Hsq: Hardware-aware automated quantization with mixed precision," 2019. [Online]. Available: <https://arxiv.org/abs/1811.08886>
- [60] J. H. Lee, J. Kim, S. J. Kwon, and D. Lee, "Flexround: Learnable rounding based on element-wise division for post-training quantization," 2024. [Online]. Available: <https://arxiv.org/abs/2306.00317>
- [61] K. T. Chitty-Venkata, J. Ye, and M. Emani, "Mopeq: Mixture of mixed precision quantized experts," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 4023–4032.
- [62] A. Clark, D. d. l. Casas, A. Guy, A. Mensch, M. Paganini, J. Hoffmann, B. Damoc, B. Hechtman, T. Cai, S. Borgeaud et al., "ST-MoE: Designing stable and transferable sparse expert models," *arXiv preprint arXiv:2202.08906*, 2022.